ISLAMIC UNIVERSITY OF TECHNOLOGY
DHAKA, BANGLADESH
ORGANIZATION OF ISLAMIC COOPERATION
DATA STRUCTURES

Name: Antara Arifa Mullick

Student ID: 210042132

Department: Computer Science and Engineering

Program: B.Sc. in Software Engineering

Quiz 4 Assignment

ARTICLE ON SEARCHING

Basic understanding of assigned topic

Searching

Searching is a fundamental operation in computer science that involves finding a specific element within a collection of data. The collection could be an array, list, tree, or any other data structure. The primary goal of searching is to locate the desired item efficiently.

Types of Searching -

1. Linear Search: Linear search, also known as sequential search, is a simple method where each element in the collection is checked one by one until the desired element is found or the entire collection is traversed. It is easy to implement and applicable to any list, regardless of whether it is sorted or unsorted.

Advantages:

- Simplicity: Easy to understand and implement.

- Applicability: Works on both sorted and unsorted lists.

Disadvantages:

- Inefficiency: Linear search can be slow for large datasets, especially if the desired element is near the end.

Time Complexity: O(n) - Linear time complexity, where 'n' is the size of the dataset.

2. Binary Search: Binary search is an efficient search algorithm that works on sorted lists. It repeatedly divides the search space in half by comparing the middle element with the target value. It is significantly faster than linear search for large sorted datasets.

Advantages:

- Efficiency: Has a time complexity of O(log n), making it highly efficient.

- Reduced Comparisons: Requires fewer comparisons compared to linear search.

Disadvantages:

- Requirement: Applicable only to sorted lists.

- Limited Applicability: Not suitable for linked lists.

Time Complexity: O (log n) - Logarithmic time complexity, where 'n' is the size of the dataset.

3. Ternary Search: Ternary search is an extension of binary search, dividing the search space into three parts instead of two. It is used on sorted lists to narrow down the search more quickly. It further optimizes the search process, making it more efficient in some scenarios than binary search.

Advantages:

- Efficiency: Offers improved efficiency over binary search in certain situations.

- Faster Convergence: May converge faster than binary search for some datasets.

Disadvantages:

- Complexity: More complex than binary search.

- Limited Applicability: Not universally applicable; effectiveness depends on the dataset.

Time Complexity: O (log3 n) - Logarithmic time complexity, where 'n' is the size of the dataset.

Operations supported

Let's have a look into the operations supported by the three types of searches: Linear Search, Binary Search, and Ternary Search.

1.Linear Search - Simple and suitable for small or unsorted datasets.

Search Method:

- Iterate through each element in the collection until the desired element is found or the end of the collection is reached.

- Compare each element with the target value.

- If a match is found, return the index of the element; otherwise, return a signal indicating that the element is not present.

Operations:

Search: Linear search performs a sequential search through the elements.

Equality Check: Compares each element with the target for equality.

2.Binary Search - Efficient for large, sorted datasets with a logarithmic time complexity.

Search Method:

- Compare the target value with the middle element of the sorted list.

- If they match, the search is successful; if not, determine whether the target is smaller or larger than the middle element.

- Repeat the process on the relevant half until the target is found or the search space is empty.

Operations:

Comparison: Repeatedly compares the target with the middle element.

Division: Divides the search space in half at each step.

3. Ternary Search - An extension of binary search, offering faster convergence in specific situations, but more complex.

Search Method:

- Compare the target value with the two midpoints that divide the sorted list into three parts.

- Based on the comparisons, narrow down the search space to one-third.

- Repeat the process until the target is found or the search space is empty.

Operations:

Comparison: Compares the target with two midpoints, dividing the search space into three parts.

Ternary Division: Divides the search space into three parts at each step.

Story/Realistic problem Scenario

Introduction:

Farhan is a diligent student who always keeps his study materials organized. He has a collection of textbooks neatly arranged on his bookshelf. One day, however, he realizes that his favorite textbook, containing crucial notes for an upcoming exam, is missing. In a panic, he decides to employ different search algorithms to find the lost book.

Linear Search Challenge:

Farhan starts with a linear search, meticulously checking each book on his shelf one by one. He carefully scans the titles, hoping to find his missing textbook. The operation is straightforward but time-consuming. After going through all the books, Farhan finally discovers his textbook buried among the last few. Linear search, while simple, proved to be a bit slow for his extensive collection.

Binary Search Adventure:

To save time, Farhan decides to organize his books alphabetically by title. With his books sorted, he employs binary search. He opens the bookshelf in the middle, checks the title, and determines whether to continue the search in the left or right half. Farhan repeats this process until he locates his textbook. Binary search significantly reduces the search time, proving especially effective for large, sorted collections.

Ternary Search Twist:

Curious about optimization, Farhan decides to try ternary search for his next challenge. He divides his bookshelf into three equal parts and checks the titles at the two breakpoints. Based on the comparisons, he narrows down the search to one-third of the shelf. This process continues until he triumphantly finds his

textbook. Ternary search, with its three-way division, proves to be a slightly faster alternative, offering a unique approach to locating items in a sorted collection.

Solution of story/Realistic problem Scenario

a) Intuition behind the solution:

The problem involves finding a lost item, namely a textbook, within a collection of items. Different search algorithms - Linear Search, Binary Search, and Ternary Search - are applied to efficiently locate the missing textbook based on the organization of the collection. The intuition is to understand how each search algorithm operates and choose the one that best suits the scenario.

b) Solution Steps:

1. Linear Search:

- Intuition: Linearly scan through the collection, comparing each item with the target until the item is found or the end of the collection is reached.

- Steps:

1. Initialize an index variable to 0.

2. Iterate through the collection until the target is found or the end is reached.

3. If the target is found, return the index; otherwise, return a signal indicating that the item is not present.

```
// C++ code for Linear Search

int linearSearch(const vector<string>& collection, const string& target) {
```

```
for (int i = 0; i < collection.size(); ++i) {
if (collection[i] == target) {
return i; // Found the target, return the index
}
}
return -1; // Target not found
}
```

2. Binary Search:

- Intuition: Divide the collection into halves and efficiently narrow down the search space by comparing the target with the middle element.

- Steps:

1. Sort the collection.
2. Initialize low and high indices.
3. Repeat until the low index is less than or equal to the high index.
4. Calculate the middle index.
5. Compare the target with the middle element.
6. Adjust the low or high index based on the comparison.

```
// C++ code for Binary Search
int binarySearch(vector<string>& collection, const string& target) {
sort(collection.begin(), collection.end()); // Sort the collection
int low = 0, high = collection.size() - 1;
```

```
while (low <= high) {
int mid = low + (high - low) / 2;

if (collection[mid] == target) {
return mid; // Found the target, return the index
} else if (collection[mid] < target) {
low = mid + 1;
} else {
high = mid - 1;
}
}

return -1; // Target not found
}
```

3. Ternary Search:

- Intuition: Divide the collection into three parts and efficiently narrow down the search space by comparing the target with two breakpoints.

- Steps:

1. Sort the collection.
2. Initialize low and high indices.
3. Repeat until the low index is less than or equal to the high index.
4. Calculate two midpoints.

5. Compare the target with the two midpoints.
6. Adjust the low or high index based on the comparisons.

```
// C++ code for Ternary Search
int ternarySearch(vector<string>& collection, const string& target) {
sort(collection.begin(), collection.end()); // Sort the collection
int low = 0, high = collection.size() - 1;

while (low <= high) {
int mid1 = low + (high - low) / 3;
int mid2 = high - (high - low) / 3;

if (collection[mid1] == target) {
return mid1; // Found the target, return the index
} else if (collection[mid2] == target) {
return mid2; // Found the target, return the index
} else if (collection[mid1] > target) {
high = mid1 - 1;
} else if (collection[mid2] < target) {
low = mid2 + 1;
} else {
low = mid1 + 1;
high = mid2 - 1;
}
}
```

return -1; // Target not found

}

c) Complexity Analysis:

Linear Search:

- Time Complexity: O(n) - Linear time, where 'n' is the size of the collection.

- Space Complexity: O(1) - Constant space.

Binary Search:

- Time Complexity: O(log n) - Logarithmic time, where 'n' is the size of the collection (due to sorting).

- Space Complexity: O(1) - Constant space (ignoring the space required for the input collection).

Ternary Search:

- Time Complexity: O(log3 n) - Logarithmic time, where 'n' is the size of the collection (due to sorting).

- Space Complexity: O(1) - Constant space (ignoring the space required for the input collection).

Conclusion Comment

Where to Use:

1. Linear Search:

- Suitable for small or unsorted datasets.
- Ideal for simplicity in tasks like searching a phone book.

2. Binary Search:

- Efficient for large, sorted datasets.
- Appropriate for tasks like finding a word in a dictionary.

3. Ternary Search:

- Useful in scenarios where a three-way division provides an advantage.
- Effective when faster convergence is required.

Where Not to Use:

1. Linear Search:

- Inefficient for large datasets.
- Less suitable for sorted data.

2. Binary Search:

- Not suitable for unsorted datasets.
- Challenging with linked lists due to the need for random access.

3. Ternary Search:

- May be complex for simple or small datasets.

- Not necessary when a two-way division (binary search) suffices.

3 other problem links of different online judge that can be solved using this article knowledge

Binary Search Problem:

[Codeforces Problem 750A - New Year and Hurry]

(https://codeforces.com/problemset/problem/750/A)

Problem Description:

Limak is going to participate in a contest on the last day of the year. The contest will start at 20:00 and last for four hours, until midnight. There will be n problems, and Limak needs k minutes to get to the New Year's Eve party. Limak wants to maximize the number of problems he can solve before the contest ends.
Implement binary search to find the maximum number of problems Limak can solve.

Binary Search Problem:

[Codeforces Problem 899/B - Sushi for Two]

(https://codeforces.com/problemset/problem/899/B)

Problem Description:

You are given a sequence of n sushi pieces, each labeled with a type (either 1 or 2). You want to choose a contiguous subsegment such that the number of 1s and 2s in the subsegment is equal and as large as possible. Implement binary search to find the maximum length of such a subsegment.

Linear Search Problem:

[Codeforces Problem 144A - Arrival of the General]
(https://codeforces.com/problemset/problem/144/A)

Problem Description:

You are given the heights of soldiers in a line. The general wants to rearrange them in such a way that the soldier with the maximum height is at the front and the one with the minimum height is at the end. Determine the minimum number of moves the general needs to make.